UFMG UNIVERSIDADE FEDERAL DE MINAS GERAIS

Seminars on Large Language Models

FLASHATTENTION: Fast and Memory-Efficient Exact Attention with IO-Awareness

Cayro Neto
Leandro Campos

Paper Identity

FLASHATTENTION: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[†], Atri Rudra[‡], and Christopher Ré[†]

[†]Department of Computer Science, Stanford University
[‡]Department of Computer Science and Engineering, University at Buffalo, SUNY

{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu,
chrismre@cs.stanford.edu

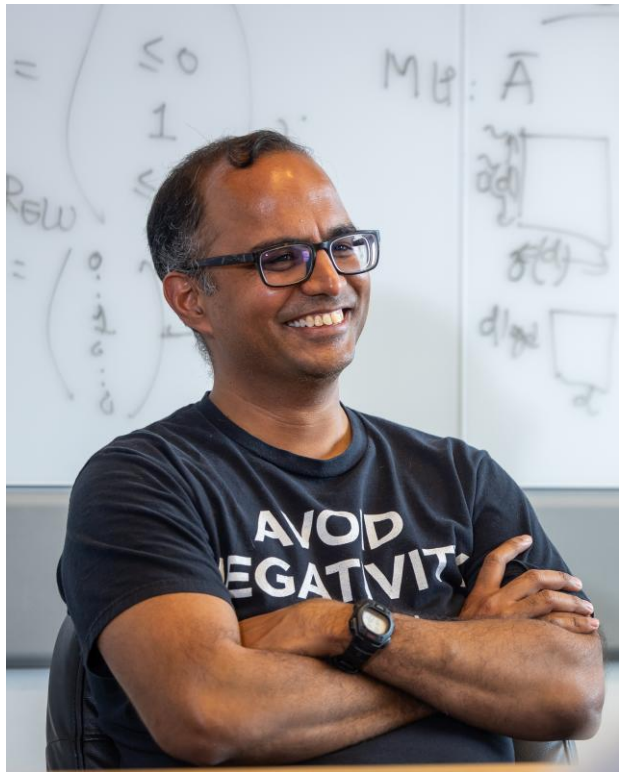
June 24, 2022

Paper Identity

Tri Dao[1] Daniel Fu[1] Stefano Ermon[1]

[1]Department of Computer Science, Stanford University

Paper Identity



Atri Rudra[2]

Christopher Re[1]

[2]Department of Computer Science and Engineering, University at Buffalo, SUNY

Paper Lineage

Online normalizer calculation for softmax

Maxim Milakov
NVIDIA
mmilakov@nvidia.com

Natalia Gimelshein
NVIDIA
ngimelshein@nvidia.com

Paper Lineage

FlashAttention-2:
Faster Attention with Better Parallelism and Work Partitioning

Tri Dao[1,2]

[1]Department of Computer Science, Princeton University
[2]Department of Computer Science, Stanford University
trid@cs.stanford.edu

July 18, 2023

FlashAttention-3:
Fast and Accurate Attention with Asynchrony and Low-precision

Jay Shah*[1], Ganesh Bikshandi*[1], Ying Zhang[2], Vijay Thakkar[3,4], Pradeep Ramani[3], and Tri Dao[5,6]

[1]Colfax Research [2]Meta [3]NVIDIA [4]Georgia Tech [5]Princeton University [6]Together AI
{jayhshah,ganesh}@colfax-intl.com, yingz@meta.com, {vithakkar,prraman}@nvidia.com, tri@tridao.me

July 16, 2024

The Problem with Standard Attention

What is Attention?

- Core mechanism in Transformers
- Calculates a *relevance score* for every pair of tokens in a sequence
- Using the relevance scores, computes a *contextualized representation* for each token

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T$$

$$\mathbf{P} = \mathrm{softmax}(\mathbf{S})$$

$$\mathbf{O} = \mathbf{P}\mathbf{V}$$

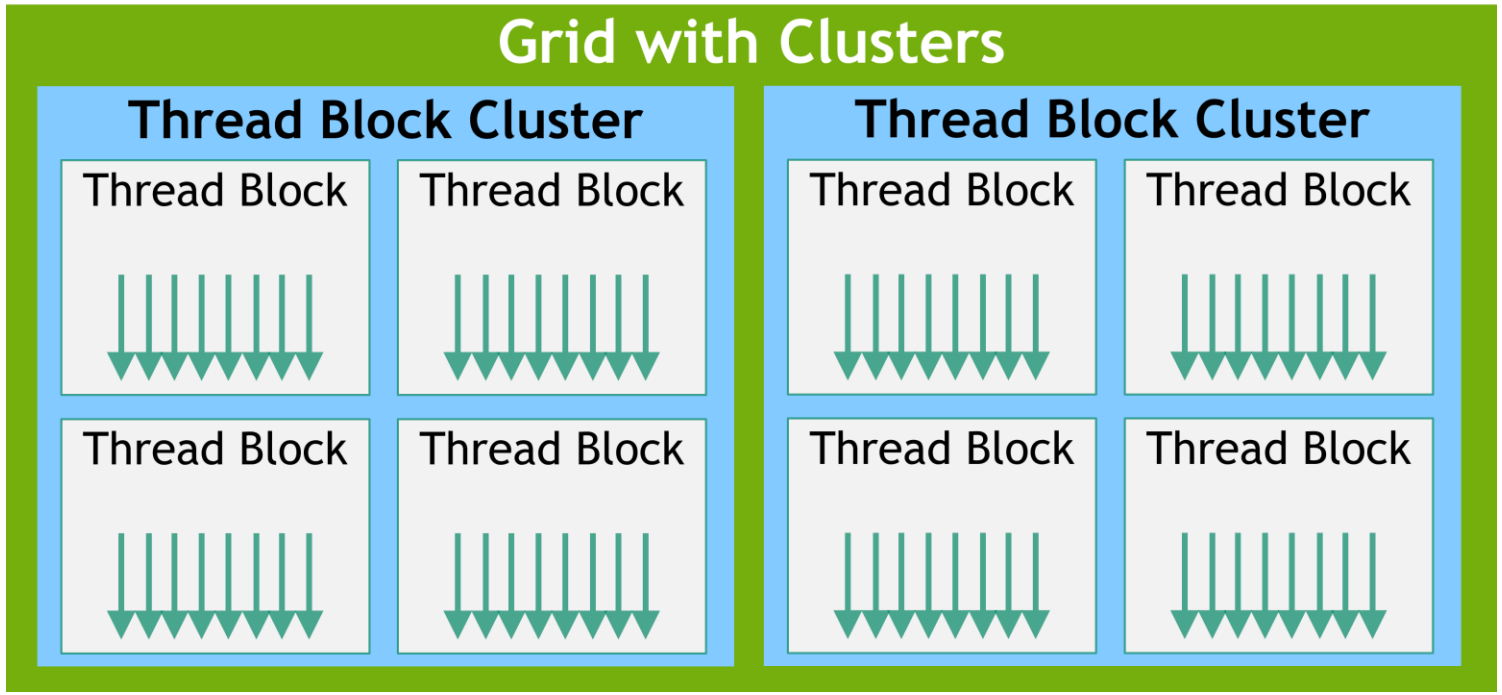
The Bottleneck

- Let N be the sequence length
- **Runtime:** $O(N^2)$
- **Memory:** $O(N^2)$
- $\mathbf{S}, \mathbf{P} \in \mathbb{R}^{N \times N}$ are stored in memory
- For FP16 *(for just one layer, one head)*

$$N = 16k \implies \mathrm{size}(\mathbf{S}) \approx 0.5\mathrm{GB}$$

$$N = 128k \implies \mathrm{size}(\mathbf{S}) \approx 32\mathrm{GB}$$

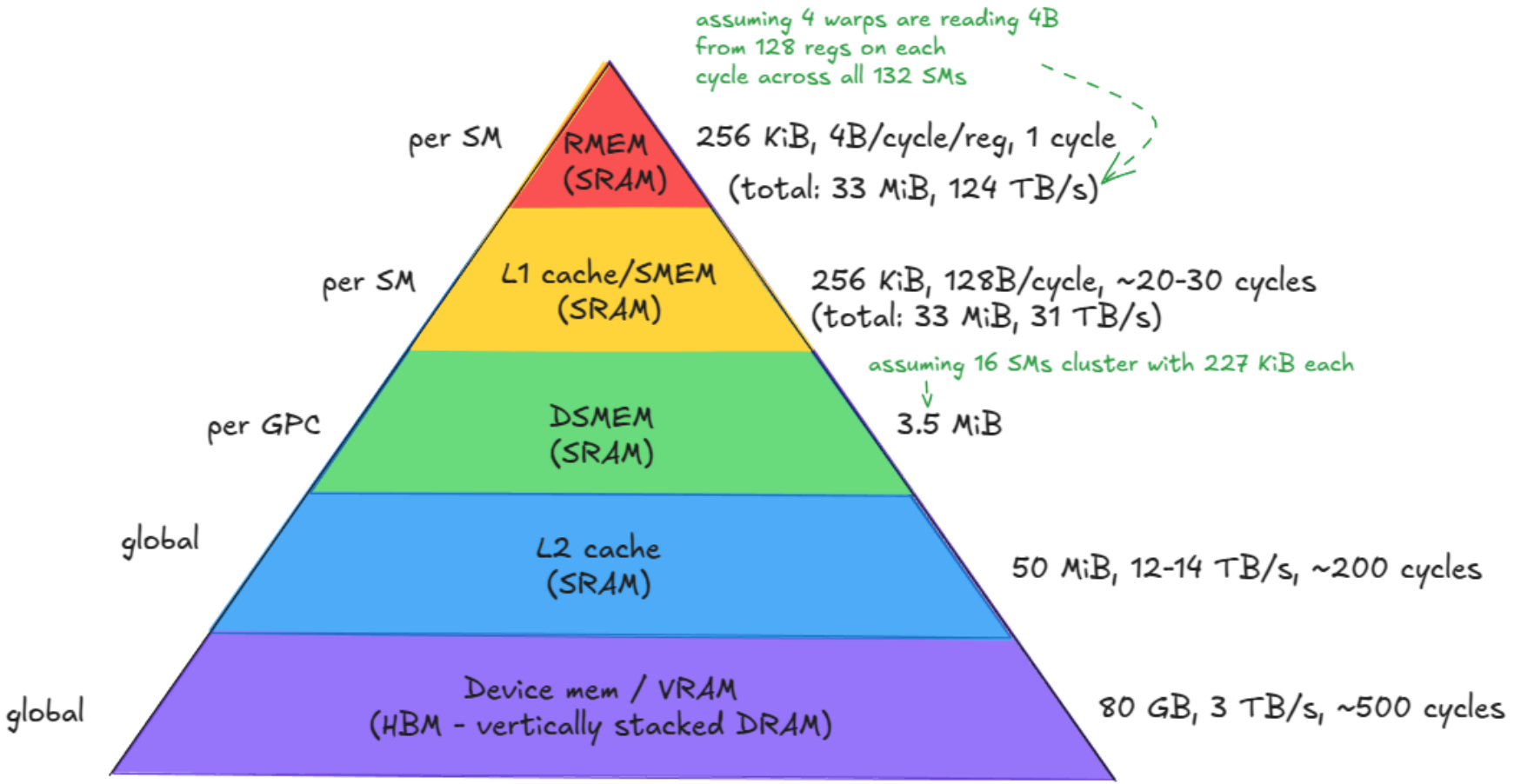
A Beginner's Guide to GPU Thread Hierarchy



Grid of Thread Block Clusters [Link]

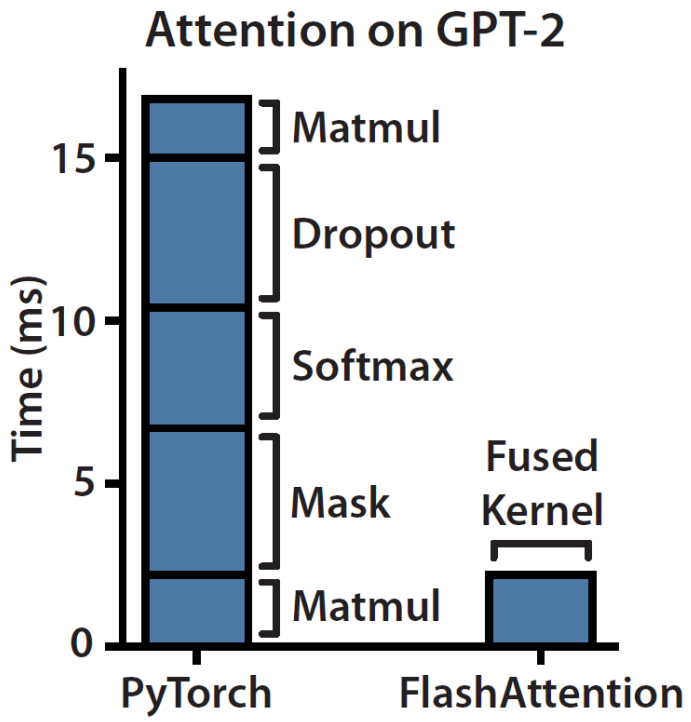
CUDA Abstraction	Hardware
Thread	Core
Warp (32 threads)	
Thread Block	Streaming Multiprocessor (SM)
Thread Block Cluster (up to 8 Thread Blocks)	
Grid	GPU

A Beginner's Guide to GPU Memory Hierarchy



Memory hierarchy of the H100 (SXM5) GPU [Link]

The Real Bottleneck: Memory I/O



Speedup over PyTorch implementation on GPT-2 [FAv1]

- Standard attention is IO-unaware
- Most of the time in mask, softmax and dropout is spent waiting for memory
- These operations are **memory-bound**
- We are bottlenecked by the HBM bandwidth, not by the GPU's ability to do math
- Can we compute exact attention without ever writing $\mathbf{S}, \mathbf{P} \in \mathbb{R}^{N \times N}$ to HBM?

Core Ideas

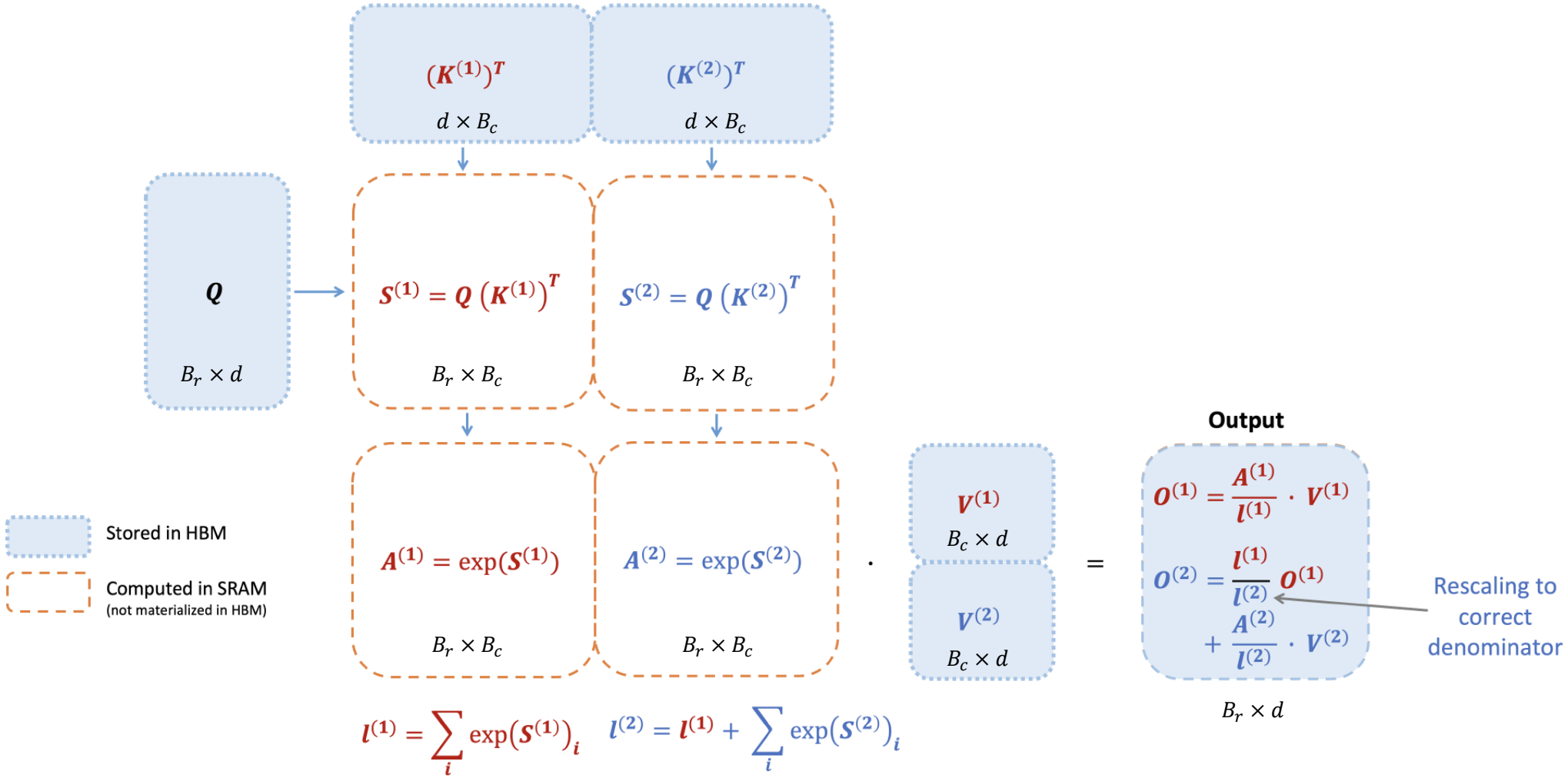
Tiling

- Load inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ from HBM in blocks
- All computation for a block happens in fast SRAM
- But… $\mathrm{softmax}$ needs the entire row to compute $\sum_j e^{x_j}$

Online Softmax

- Compute $\mathrm{softmax}$ incrementally during a single pass over the blocks
- Keep track of *running* $\mathrm{softmax}$ statistics
- As new blocks arrive, rescale previous results and update statistics
- **Result:** Exact same answer

FLASHATTENTION v1: Forward Pass (The Algorithm)



Online Softmax

For simplicity, consider just one row block of the attention matrix $\mathbf{S}$, of the form $\mathbf{S} = [\mathbf{S}^{(1)} \quad \mathbf{S}^{(2)}]$ for some matrix $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$, where B_r and B_c are the row and the column block sizes.

$$
\begin{aligned}
m^{(1)} &= \mathrm{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \mathrm{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
\tilde{\mathbf{P}}^{(1)} &= \mathrm{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\
\mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)} \mathbf{V}^{(1)} = \mathrm{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \mathrm{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \mathrm{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \mathrm{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \mathrm{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \mathrm{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
\mathbf{O}^{(2)} &= \mathrm{diag}(\ell^{(1)} / \ell^{(2)})^{-1} \mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)} \mathbf{V}^{(2)} = \mathrm{diag}(\ell^{(2)})^{-1} e^{s^{(1)} - m} \mathbf{V}^{(1)} + \mathrm{diag}(\ell^{(2)})^{-1} e^{s^{(2)} - m} \mathbf{V}^{(2)} = \mathbf{O}.
\end{aligned}
$$

Online Softmax

For simplicity, consider just one row block of the attention matrix $\mathbf{S}$, of the form $\mathbf{S} = [\mathbf{S}^{(1)} \quad \mathbf{S}^{(2)}]$ for some matrix $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$, where B_r and B_c are the row and the column block sizes.

$$m^{(1)} = \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r}$$

$$\ell^{(1)} = \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r}$$

$$\tilde{\mathbf{P}}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - m^{(1)}} \in \mathbb{R}^{B_r \times B_c}$$

$$\mathbf{O}^{(1)} = \tilde{\mathbf{P}}^{(1)} \mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d}$$

$$m^{(2)} = \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m$$

$$\ell^{(2)} = e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell$$

$$\tilde{\mathbf{P}}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}}$$

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)} / \ell^{(2)})^{-1} \mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)} \mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{s^{(1)} - m} \mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{s^{(2)} - m} \mathbf{V}^{(2)} = \mathbf{O}.$$

$$\text{diag}(\ell^{(1)} / \ell^{(2)}) e^{m^{(1)} - m^{(2)}} \mathbf{O}^{(1)}$$

Backward Pass & Block-Sparse

Backward Pass

- **Problem:** The backward pass needs $\mathbf{S}, \mathbf{P}$ we just threw away
- **Solution:** Recomputation
- We saved the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$
- During the backward pass, we just recompute the blocks of $\mathbf{S}, \mathbf{P}$ in SRAM again
- It is faster to do extra math in SRAM than to read the huge matrices from HBM

Block-Sparse

- The same IO-aware principle can be applied to approximate attention
- This makes sparse attention actually fast, which other methods failed to do because they were still **memory-bound**

Experimental Results

Faster Models with FLASHATTENTION

Table 1: Training time of BERT-large, starting from the same initialization provided by the MLPerf benchmark, to reach the target accuracy of 72.0% on masked language modeling. Averaged over 10 runs on 8×A100 GPUs.

BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FLASHATTENTION (ours)	**17.4** ± 1.4

Table 2: GPT-2 small and medium using FLASHATTENTION achieve up to 3× speed up compared to Huggingface implementation and up to 1.7× compared to Megatron-LM. Training time reported on 8×A100s GPUs.

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FLASHATTENTION	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FLASHATTENTION	14.3	**6.9 days (3.0×)**

Faster Models with FLASHATTENTION

Table 3: The performance of standard attention, FLASHATTENTION, block-sparse FLASHATTENTION, and approximate attention baselines on the Long-Range-Arena benchmarks.

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FLASHATTENTION	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
Block-sparse FLASHATTENTION	37.0	63.0	81.3	43.6	73.3	59.6	**2.8**×
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attention [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Local Attention [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

Better Models with Longer Sequences

Table 4: GPT-2 small with FlashAttention, with 4× larger context length compared to Megatron-LM, is still 30% faster while achieving 0.7 better perplexity. Training time on 8×A100 GPUs is reported.

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FlashAttention	1k	18.2	**2.7 days (1.7×)**
GPT-2 small - FlashAttention	2k	17.6	3.0 days (1.6×)
GPT-2 small - FlashAttention	4k	**17.5**	3.6 days (1.3×)

Better Models with Longer Sequences

Table 5: Long Document performance (micro F_1) at different sequence lengths using FlashAttention.

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	**57.1**
ECtHR [6]	72.2	74.3	77.1	78.6	**80.7**	79.2

Table 6: We report the first Transformer model that can achieve non-random performance on Path-X and Path-256.

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention	**61.4**	✗
Block-sparse FlashAttention	56.0	**63.1**

Benchmarking Attention

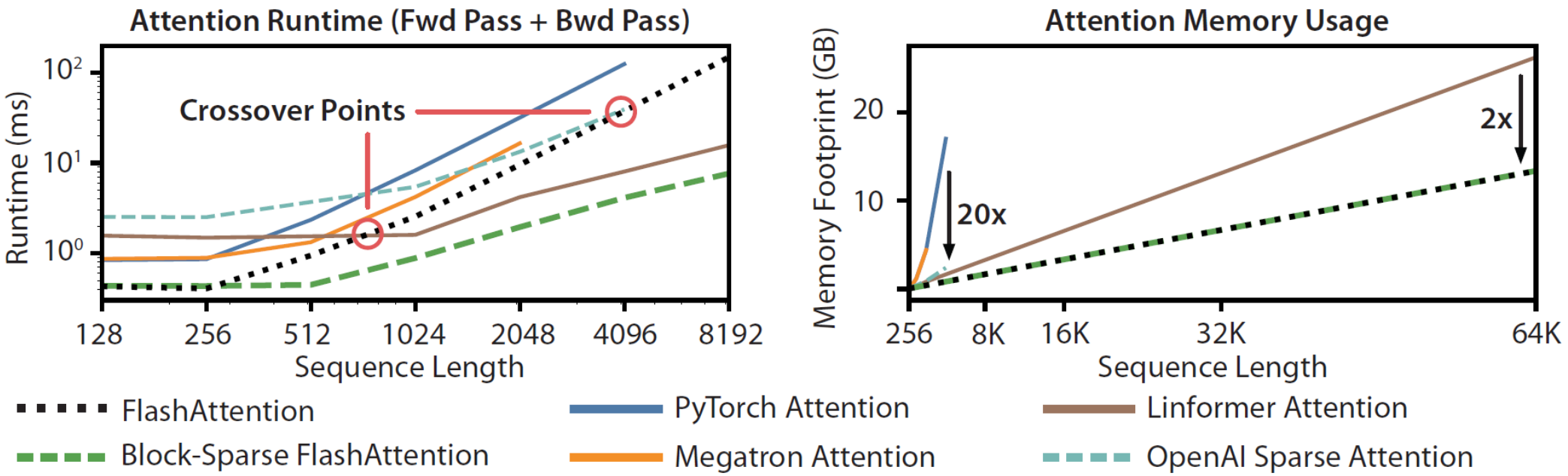


Figure 3: **Left:** runtime of forward pass + backward pass. **Right:** attention memory usage.

The Evolution

FLASHATTENTION-2 (A100 Focus)

- **Goal:** Close the gap to peak FLOPs (v1 ≈ 25-40% of peak)
- **How:**
  - Parallelize over sequence length (N) dimension
  - Switch internal work partitioning to "split-Q" (vs "split-K" in v1) to reduce shared-memory traffic
- **Result:** ∼2× faster than v1; Reached 50–73% of A100 peak FLOPs/s.

FLASHATTENTION-3 (H100 Focus)

- **Goal:** Exploit H100 asynchronous features (v2 ≈ 35% utilization on H100)
- **How:**
  - **Asynchrony:** Overlap memory (TMA) & compute (WGMMA) via warp specialization (producer/consumer)
  - **Pipelining:** Overlap slow $\mathrm{softmax}$ operations with asynchronous GEMMs
  - **FP8 Support:** Leverage FP8 Tensor Cores with block quantization & incoherent processing for accuracy
- **Result:** ∼ 1.5–2.0× faster than v2 on H100; Up to ∼740 TFLOPs/s (≈75% peak) in FP16; ∼1.2 PFLOPs/s in FP8

Limitations and Future Directions

Limitations and Future Directions

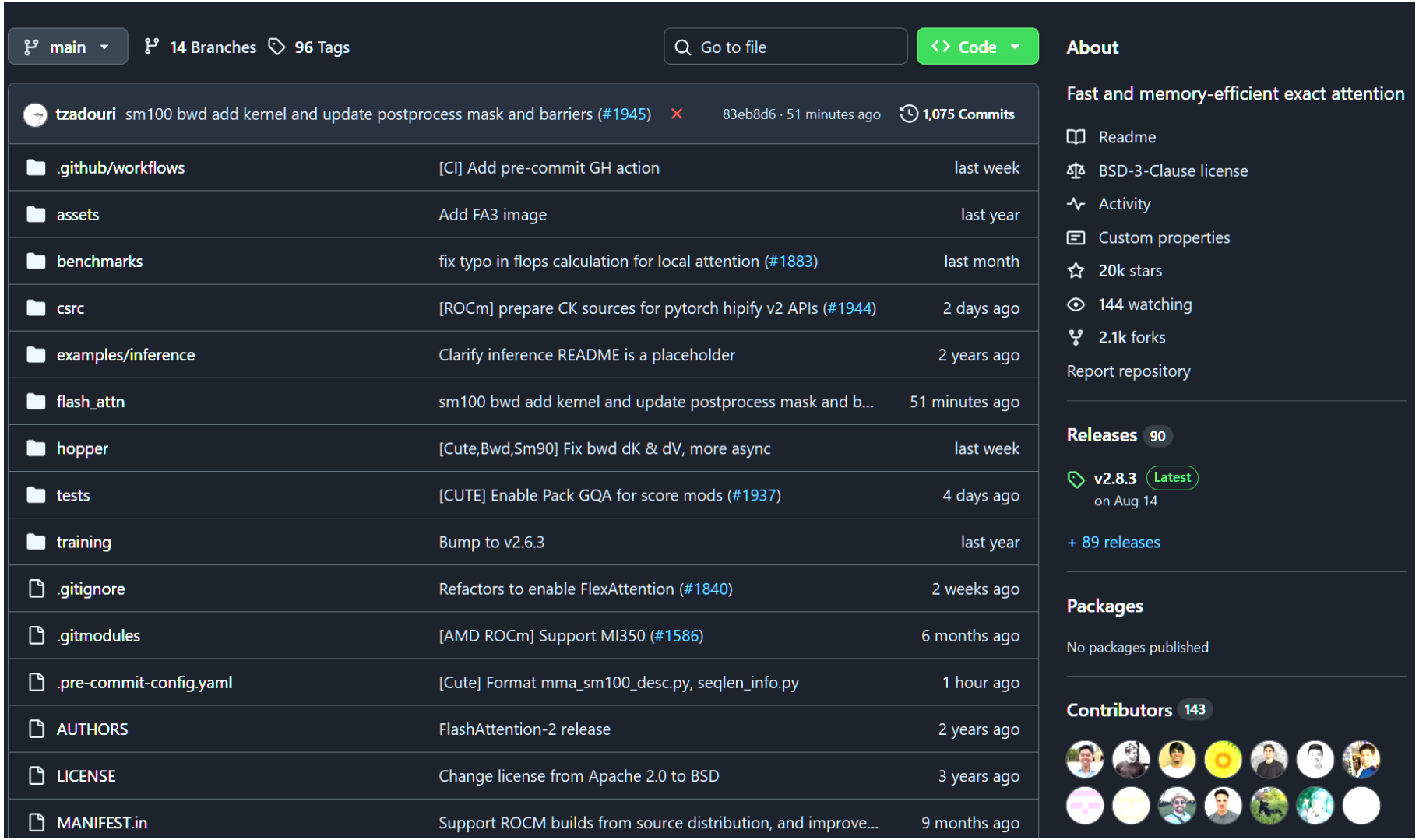
IO-Aware Deep Learning: The IO-aware approach could extend beyond attention, the most memory-intensive Transformer computation.

Compiling to CUDA: Current IO-aware attention implementations require a new, low-level CUDA kernel for each, demanding significant effort and limiting transferability.

Multi-GPU IO-Aware Methods: Attention can be parallelized across multiple GPUs, which adds an additional layer to IO analysis: accounting for data transfer between GPUs.

Societal Impacts: Addressing risks like misinformation (language modeling) and automatic surveillance (image classification) requires tackling application-specific issues such as privacy, bias, and discrimination.

Paper Implementation



https://github.com/Dao-AILab/flash-attention/